

IA conversationnelle & automatisation documentaire

Conception et implémentation — Optimi Santé.

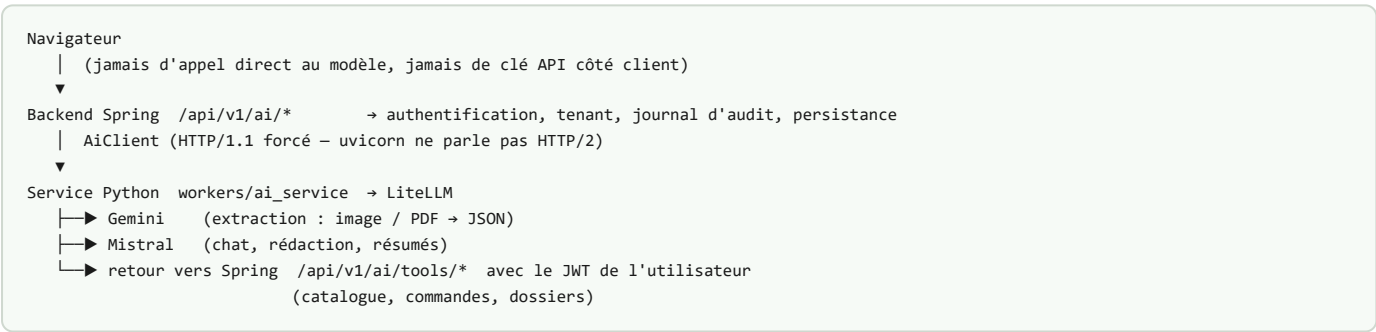
4 septembre 2026 • Modèles retenus : **Gemini** (lecture de documents) et **Mistral** (assistant, rédaction, résumés) • **Aucun appel à un modèle premium** • Architecture volontairement portable.

1. Choix des modèles et portabilité

Usage	Modèle	Pourquoi
Extraction documentaire (passeport, diplôme, inscription à l'Ordre...)	<code>gemini/gemini-2.0-flash</code>	Lit nativement images et PDF — aucun moteur OCR à installer. Palier gratuit réel, puis quelques centimes.
Assistant conversationnel, rédaction, résumés	<code>mistral/mistral-small-latest</code>	Hébergement UE (argument RGPD pour une plateforme santé), coût faible, bon support du <i>function calling</i> .
Embeddings (si RAG un jour)	<code>mistral/mistral-embed</code>	Déjà configuré, non utilisé pour l'instant (voir \$7).

Un seul fichier connaît un fournisseur : `app/llm.py` . Tout passe par **LiteLLM**, qui expose un appel unique `completion(model="<fournisseur>/<modèle>", ...)` . Changer de modèle ou de fournisseur — Groq, Together, OpenRouter, Ollama en local, ou un modèle auto-hébergé — se fait en modifiant **une variable d'environnement** (`EXTRACTION_MODEL` , `CHAT_MODEL` , `DRAFT_MODEL`), sans toucher au code Python, au backend Java ni au frontend.

2. Architecture



Le service Python est le **jumeau du worker de reporting** déjà en place : même patron de passerelle, même gestion de l'indisponibilité, même correctif HTTP/1.1 découvert lors du Sprint 5.

2.1 Sécurité — trois garanties

Garantie	Mise en œuvre
Le modèle ne voit que ce que l'utilisateur a le droit de voir	Les outils du chat rappellent le backend avec le JWT de l'utilisateur : contexte multi-tenant, rôles et filtre de suppression logique (<code>@SQLRestriction</code>) restent appliqués. Le service IA n'interroge jamais la base pour des données personnelles.
L'IA ne modifie jamais un dossier	L'extraction renvoie une proposition . La colonne <code>applied_at</code> n'est renseignée que lorsqu'un humain a relu et validé. Exigence RGPD autant que précaution métier.
Tout est tracé	Chaque analyse, chaque brouillon et chaque validation passent par <code>AuditService</code> (actions <code>AI_DOCUMENT_EXTRACT</code> , <code>AI_DOCUMENT_APPLIED</code> , <code>AI_DRAFT</code>). Les conversations sont historisées.

Le prompt système interdit explicitement le conseil médical, le diagnostic, la posologie, la divulgation de données d'un tiers et toute demande de mot de passe ou de coordonnées bancaires.

3. Automatisation documentaire (Gemini)

8 types de documents alignés sur l'énumération `DocumentType` du dossier CHU : passeport, diplôme, inscription à l'Ordre, garantie financière, visa, lettre consulaire, justificatif d'hébergement, autre.

Ce que fait l'analyse	Détail
Extraction champ par champ	Ex. passeport : nom, prénoms, n°, nationalité, dates de naissance / délivrance / expiration, pays émetteur, bande MRZ. Dates normalisées en <code>AAAA-MM-JJ</code> .
Indice de confiance par champ	0 à 1. Les champs sous 60 % sont surlignés en orange dans l'écran de validation : l'agent sait quoi relire en priorité.
Contrôle de cohérence de type	<code>type_matches</code> : le modèle signale si le fichier déposé n'est pas du type annoncé (un diplôme envoyé à la place d'un passeport, par exemple).
Anomalies	Liste d'alertes : document expiré, illisible, raturé, informations contradictoires.
Zéro invention	Consigne stricte : un champ illisible ou absent vaut <code>null</code> avec une confiance de 0, jamais une valeur devinée.

Formats acceptés : PDF, JPEG, PNG, WEBP, HEIC, TIFF — 15 Mo maximum. Aucun fichier n'est conservé par le service IA ; seule la trace de l'analyse est enregistrée.

4. Rédaction assistée & résumés (Mistral)

Type	Usage
<code>ENROLLMENT_DECISION</code>	E-mail de décision sur un dossier CHU (acceptation / refus)
<code>PARTNERSHIP_DECISION</code>	E-mail de décision sur une demande de partenariat
<code>DOCUMENT_REQUEST</code>	Demande de pièces manquantes, listées avec le motif
<code>CONVENTION_INTRO</code>	Préambule de convention tripartite (alimente le générateur PDF existant)
<code>TRAINING_DESCRIPTION</code>	Description publique d'une formation + objectifs pédagogiques
<code>GENERIC_EMAIL</code>	E-mail libre à partir d'une intention
Résumés : dossier CHU, demande de partenariat, commande — en 3 à 5 puces actionnables.	

Le service produit du **texte**, jamais un PDF : la mise en page reste au moteur Thymeleaf existant (`PdfGeneratorService`). Les informations absentes du contexte sont laissées **entre crochets** plutôt qu'inventées. Le texte s'ouvre dans une fenêtre où l'administrateur le relit, le modifie, puis le copie ou l'insère.

5. Assistant conversationnel (Mistral)

Aspect	Détail
Disponibilité	Bouton flottant sur tous les espaces , y compris hors connexion (le bouton WhatsApp de la page d'accueil est conservé, l'assistant se place au-dessus).
Outils	<code>search_catalog</code> , <code>list_trainings</code> (publics) ; <code>get_my_orders</code> , <code>get_my_enrollments</code> (connecté). Le jeu d'outils dépend du rôle.
Personnalisation	Prompt système adapté au rôle (médecin, client B2C/B2B, partenaire, admin, visiteur) et suggestions de départ correspondantes.
Réponse en flux	SSE de bout en bout : Python → Spring → navigateur. Le texte s'affiche au fil de la génération.
Historique	Conversations et messages persistés (<code>ai_conversations</code> , <code>ai_messages</code>), reprise du fil, traçabilité des réponses données.
Garde-fous	Pas de conseil médical, pas de données tierces, pas de prix ou de statut inventé (obligation de passer par un outil), question limitée à 4 000 caractères.

6. Fichiers

6.1 Service Python — workers/ai_service/ (nouveau)

Fichier	Rôle
app/llm.py	Seul fichier qui connaît un fournisseur. LiteLLM, blocs multimodaux, mode JSON robuste (tolère les réponses encadrées de ``), streaming, boucle d'outils.
app/schemas.py	Les 8 types de documents et leurs champs.
app/extraction.py	Prompt d'extraction, appel Gemini, validation Pydantic, normalisation des confiances.
app/drafting.py	6 types de brouillons + 4 types de résumés.
app/chat.py	Prompt système par rôle, rondes d'outils, réponse streamée.
app/tools.py	Définition des outils et rappel du backend avec le JWT.
app/settings.py	Configuration, détection « fournisseur non configuré ».
app/main.py	API FastAPI : /health , /extract , /draft , /summarize , /chat , /chat/stream .
Dockerfile , requirements.txt , .env.example , README.md	Packaging et documentation.

6.2 Backend Java — domain/ai/ (nouveau)

Fichier	Rôle
service/AiClient.java	Passerelle HTTP/1.1 vers le service Python (multipart, JSON, relais SSE), messages d'indisponibilité explicites.
service/AIService.java	Orchestration, persistance des conversations et des analyses, audit, markApplied .
web/AiResource.java	API front : statut, extraction, brouillons, résumés, chat, chat en flux (SseEmitter), historique.
web/AiToolResource.java	Outils de l'assistant, passant par la chaîne de sécurité normale.
entity/ + repository/	AiConversation , AiMessage , AiDocumentJob et leurs dépôts.
db/migration/V25__ai_assistant.sql	3 tables + index.
Modifiés : application.yml (app.ai.worker-base-url), SecurityConfig (chat et outils publics ouverts aux visiteurs, données personnelles toujours protégées), docker-compose.yml (service ai sur 8200), docker/.env (clés + choix des modèles).	

6.3 Frontend (nouveau)

Fichier	Rôle
api/aiService.ts	Client complet + streamChat en fetch / ReadableStream (l' EventSource ne sait pas envoyer d'entête Authorization) + libellés lisibles des champs.
components/ai/ChatWidget.tsx	Bouton flottant + panneau, réponse en flux, suggestions par rôle, gestion d'erreur.
components/ai/DocumentExtractionPanel.tsx	Dépôt du document, champs pré-remplis éditables, confiance affichée, alertes, bouton « Valider et reporter ».
components/ai/AiDraftButton.tsx	Bouton « ✨ » réutilisable : génère, laisse relire et modifier, régénère, copie ou insère.
pages/admin/AdminAiPage.tsx	Écran /admin/ai : état du service et des clés, analyse de document, essai des brouillons.
Modifiés : App.tsx (route /admin/ai + widget global), layouts/AdminLayout.tsx (entrée de menu « Intelligence artificielle »).	

7. Coût et évolutions

Poste	Estimation mensuelle
Extraction (~1 000 documents)	≈ 1 \$ — voire 0 \$ dans le palier gratuit Gemini
Chat (~2 000 conversations)	≈ 7 \$
Brouillons et résumés	< 1 \$

Total	≈ 10 \$/mois (30–40 \$ si le volume triple). Aucun abonnement : paiement à l'usage, plafond réglable sur le tableau de bord du fournisseur.
-------	---

RAG : pas nécessaire aujourd'hui. Le catalogue tient largement dans le contexte du modèle, et l'assistant interroge les données réelles par ses outils plutôt que par une base vectorielle. Le jour où le corpus grossit (politiques longues, historique de support, dizaines de formations), la brique est prévue : `pgvector` sur le PostgreSQL existant (image à passer en `pgvector/pgvector:pg16`) + `mistral-embed` ou un modèle d'embeddings local — aucune infrastructure supplémentaire.

8. Vérifications

Élément	Résultat
Compilation backend (JDK 24 → cible 21)	✅ BUILD SUCCESS , 0 erreur (JDK 24 → cible 21). Deux correctifs en cours de route, voir §8.1.
Build frontend (<code>tsc -b</code> + <code>vite build</code>)	✅ <code>tsc -b</code> 0 erreur, <code>vite build</code> OK ; serveur de développement relancé, les modules IA sont servis.
Image Docker du service IA	✅ <code>docker-ai:latest</code> , 611 Mo. Le premier build échouait (<code>litellm</code> exige <code>httpx<0.28</code> alors que la version était figée à 0.28.1) — bornes assouplies.
Migration <code>V25__ai_assistant.sql</code>	✅ « <i>Migrating schema to version 25 - ai assistant ... Successfully applied 1 migration</i> ». Les 3 tables <code>ai_conversations</code> , <code>ai_messages</code> , <code>ai_document_jobs</code> existent.
Service IA <code>/health</code>	✅ <code>GET :8200/health</code> → 200, les 3 capacités annoncées avec leur modèle, les 8 types de documents listés. Backend démarré en 44 s sans erreur.
Mode dégradé (sans clé API)	✅ Sans clé, tout reste debout et les messages sont actionnables : • service Python : <code>503</code> « Le fournisseur « gemini » n'est pas configuré : renseignez GEMINI_API_KEY... » • via le backend : <code>409</code> « Fonction IA indisponible : clé API du fournisseur non configurée... » • validations : <code>415</code> sur un type de fichier refusé (formats listés), <code>400</code> sur une question vide.
Outils publics de l'assistant	✅ <code>/ai/tools/trainings</code> en anonyme → 200 avec les 3 formations réelles (spécialité, prix, durée). ✅ <code>/ai/tools/catalog/search?q=chirurgie</code> → 200, 1 résultat pertinent. ✅ <code>/ai/tools/my-orders</code> authentifié → 200 ; en anonyme → 401 : les données personnelles restent protégées. ✅ Chat en anonyme : la route publique est bien atteinte (409 dégradé, et non 401).
Recherche insensible aux accents	✅ Après correctif (§8.1 n°3), vérifié sur la base réelle : • <code>echographie</code>, <code>ECHOGRAPHIE</code> et <code>Échographie</code> → tous les trois renvoient « Échographie clinique appliquée » ; • <code>pediatrie</code> → « Réanimation néonatale » (spécialité <i>PÉDIATRIE</i>) ; <code>neonatale</code> → la même, trouvée par son titre accentué ; • <code>urgence</code> → <code>[]</code>, aucune correspondance inventée.

8.1 Défauts trouvés en testant et corrigés

#	Symptôme	Cause & correctif
1	Image <code>ai</code> impossible à construire — <code>ResolutionImpossible</code>	<code>litellm</code> exige <code>httpx<0.28</code> alors que <code>httpx==0.28.1</code> était figé. Versions remplacées par des bornes : on épingle ce qu'on maîtrise, on laisse LiteLLM imposer ses propres contraintes.
2	Backend en boucle de redémarrage après ajout du domaine IA	<code>AIService</code> injectait un bean <code>ObjectMapper</code> que ce contexte Spring n'expose pas (« <i>No qualifying bean of type ObjectMapper</i> »). Remplacé par une instance locale — l'unique usage est la sérialisation des champs extraits vers une colonne TEXT.
3	La recherche catalogue ne trouvait rien sur « echographie »	Comparaison sensible aux accents : « Échographie » en base ne correspondait pas à la saisie sans accent — cas courant sur une plateforme francophone. Ajout d'une normalisation <code>TRANSLATE</code> des deux côtés, sans dépendre de l'extension PostgreSQL <code>unaccent</code> (non installée). Correctif vérifié — voir §8.
4	Première version du correctif n°3 : backend impossible à construire	J'avais tenté d'insérer une expression au milieu d'un <i>bloc de texte</i> Java, ce que le langage n'autorise pas (« <i>illegal text block open delimiter sequence</i> »). Les deux requêtes ont été réécrites en constantes assemblées par concaténation.

Deux pièges d'outillage rencontrés, notés pour la suite : un `| tail` derrière un `docker compose build` renvoie le code de sortie de `tail`, pas celui du build — un échec est alors rapporté comme un succès ; et la compilation *incrémentale* de Maven peut renvoyer `BUILD SUCCESS` sur un fichier cassé si les classes compilées précédentes sont encore là. Les vérifications suivantes capturent le code de sortie réel et forcent une recompilation du paquet concerné.

9. Mise en service

- Créer les clés : **Gemini** sur `aistudio.google.com/apikey` (gratuit), **Mistral** sur `console.mistral.ai/api-keys`.
- Les renseigner dans `docker/.env` (`GEMINI_API_KEY`, `MISTRAL_API_KEY`) — ce fichier n'est jamais versionné.
- `docker compose up -d --no-deps ai`, puis vérifier sur `/admin/ai` que les deux capacités passent au vert.

Sans clé, la plateforme fonctionne normalement : les fonctions IA s'annoncent désactivées avec un message explicite, rien d'autre n'est bloqué.

Document généré le 4 septembre 2026 — Optimi Santé, IA conversationnelle & automatisation documentaire.